Bag of Words

Machine Learning on Text

- Spam Filter:

Greetings,

I seek for a trusted, sincere and honest person to judiciously set up a charity foundation under my sponsorship.

The foundation will be strictly for helping the poorest of the poor; especially the orphanage homes and homes for destitute and the mentally retarded.

I am 76 years old and suffering from Protracted Cancer Of The Lungs and on admission at the specialist hospital intensive care unit.

My lawyer that was supposed to handle the process of executing my Will & Testament was shot during the protest against the presidential election.

If t touched your spirit to be part in putting smile and new life on the vulnerable, kindly reply through my personal below email for more information.

Mrs. Matsushita

Email: matsumorand@gmail.com

Formulaire sans titre

REMPLIR LE FORMULAIRE

Machine Learning on Text

- Spam Filter:



JF

info5@mk-yw.top on behalf of Joy Fu <info@hizbao.com>
Fri 10/30/2020 2:38 PM
To: Kasper Green Larsen

Dear ,

We are more than 10 years walkie talkie and accessories manufacturer in China. Our product cover POC radio, analog radio, DMR radio, radio headset, antenna, battery and so on.

If you need these products, please reply us with your requirement detials! We will offer you our best price!

Waiting for your early reply!

Thanks and best regards!

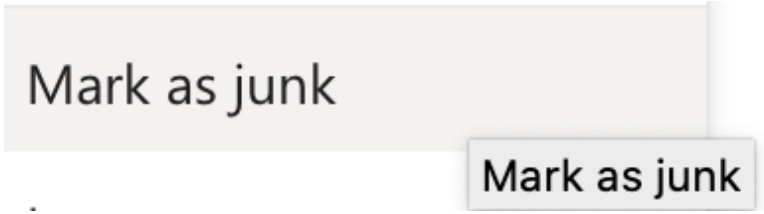
Hizbao

Reply | Forward

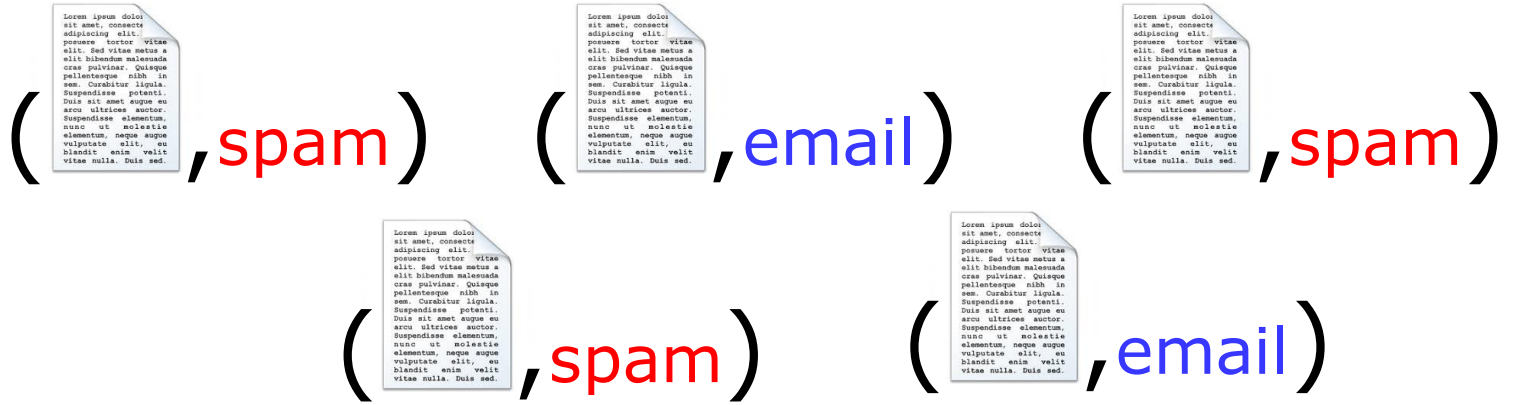
Machine Learning on Text

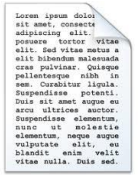
- Spam Filter:
  - Spam or real email?

- Supervised Learning:
  - Have lots of labeled data.
  - Users report spam.



- Data Set:



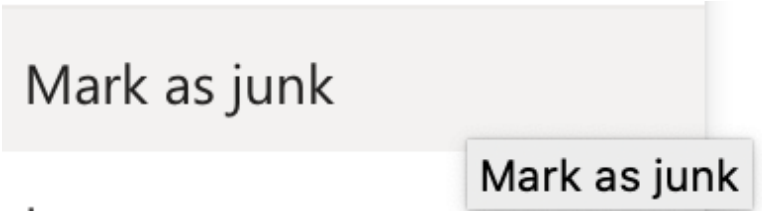


Spam or email?

Machine Learning on Text

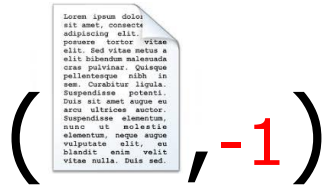
- Spam Filter:
  - Spam or real email?

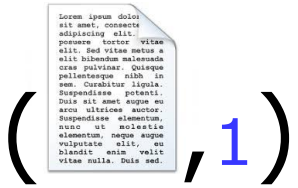
- Supervised Learning:
  - Have lots of labeled data.
  - Users report spam.

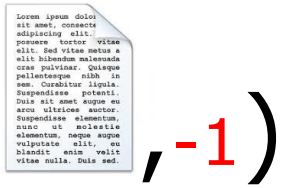


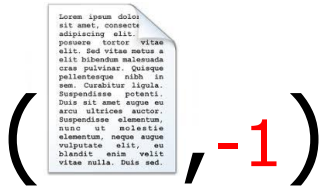
- Data Set:

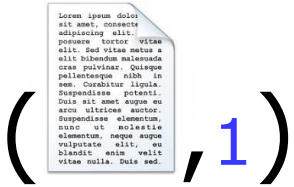
Labels we know how to handle

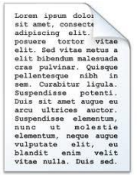








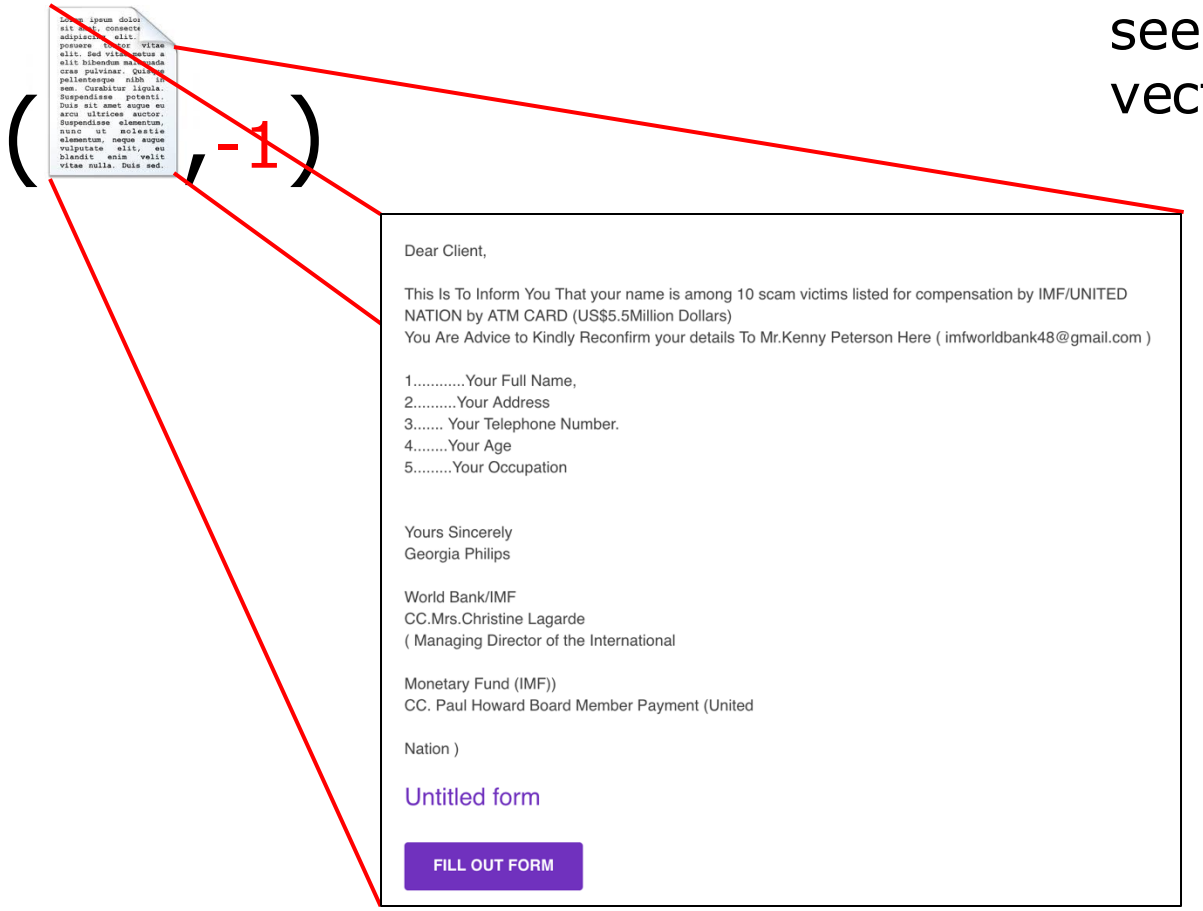




-1 or 1?

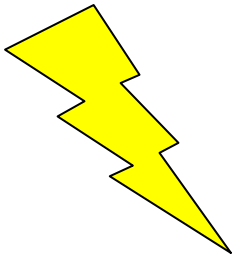
Machine Learning on Text

- Problem:



Not a feature vector!

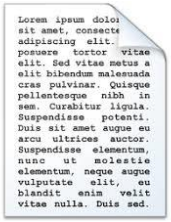
All algorithms we have seen expect feature vectors.



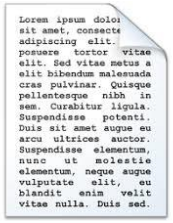
Do we need to make new algorithms just for text?

Machine Learning on Text

- Problem: Text is not a feature vector.



- Solution: Find vector representations of text.
  - Embed text/strings to vectors in R^d for some d.



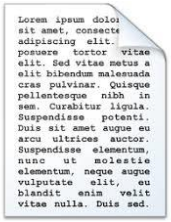


(0,1,1,0,…,2,0,1,0)

  - Then we can re-use all the algorithms we have seen so far.

Machine Learning on Text

- Goal: Represent text as a vector.



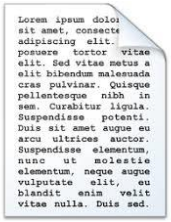
- Hope: Vector representation has all the information about the text needed for learning task.

- Simplest Representation:
  - Bag of Words.



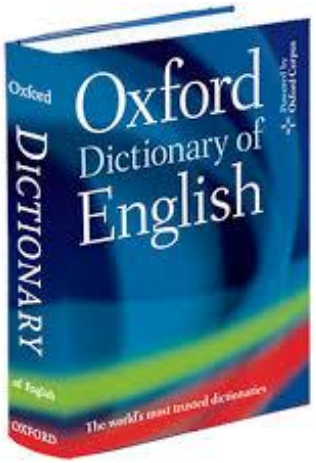
Bag of Words

- Goal: Represent text as a vector.

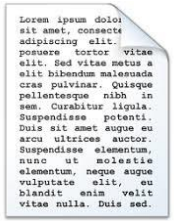


"Viagra Oral Jelly is a remedy that helps to control and restore potency during sexual arousal. The main advantage of Viagra Oral Jelly is its fast action"

- Bag of Words:
  - Assume all words in text come from the English dictionary.



Vector has 1 entry per word in dictionary





"Viagra"

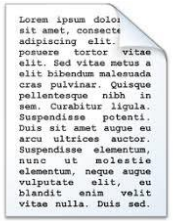
(0,1,0,0,…,2,0,1,0)

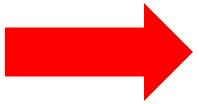
"potency" "cow"

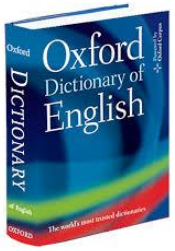
Entry stores count of many times word occurs in text.

Bag of Words

- Bag of Words:
  - Assume all words in text come from the English dictionary.







(0,1,0,0,…,2,0,1,0)

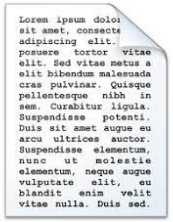
- Another Example:
  - *"partner abroad who can accommodate 200M USD for Investment"*

(0,….,0,1,0,…,0,1,……..,0,0,1,0,0,……..,0,1,0,….)

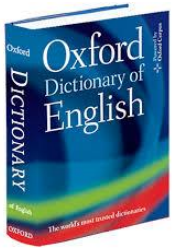
who USD investment abroad

Bag of Words

- Bag of Words:
  - Assume all words in text come from the English dictionary.





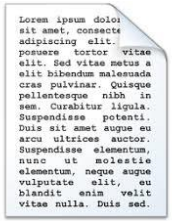


(0,1,0,0,…,2,0,1,0)

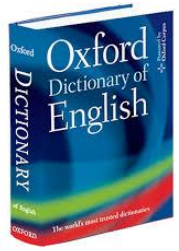
- Observations:
  - Documents with mostly the same words have similar vectors.
  - Example words mostly in spam: ”viagra”, ”cheap”, ”donation”, ”investment”, ”USD”,…
  - Reasonable that representations can be used for classification.

Bag of Words

- Bag of Words:
  - Assume all words in text come from the English dictionary.







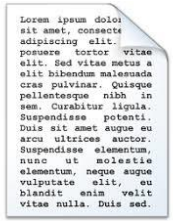
(0,1,0,0,…,2,0,1,0)



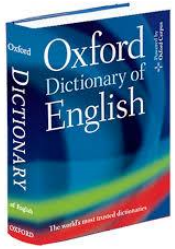
- Observations:
  - Ignores order of words in input text.
    - ”She does not like Trump. She likes Biden.”
    - ”She likes Trump. She does not like Biden.”
  - Same bag of words representation, not same meaning.

Bag of Words

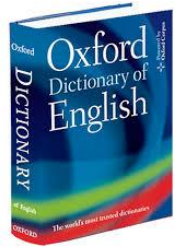
- 2-Grams:
  - Adding order to words.
  - Group together every two consecutive words.
  - Feature for *every pair* of words in dictionary.



Counts how many times two consecutive words occur in text



X



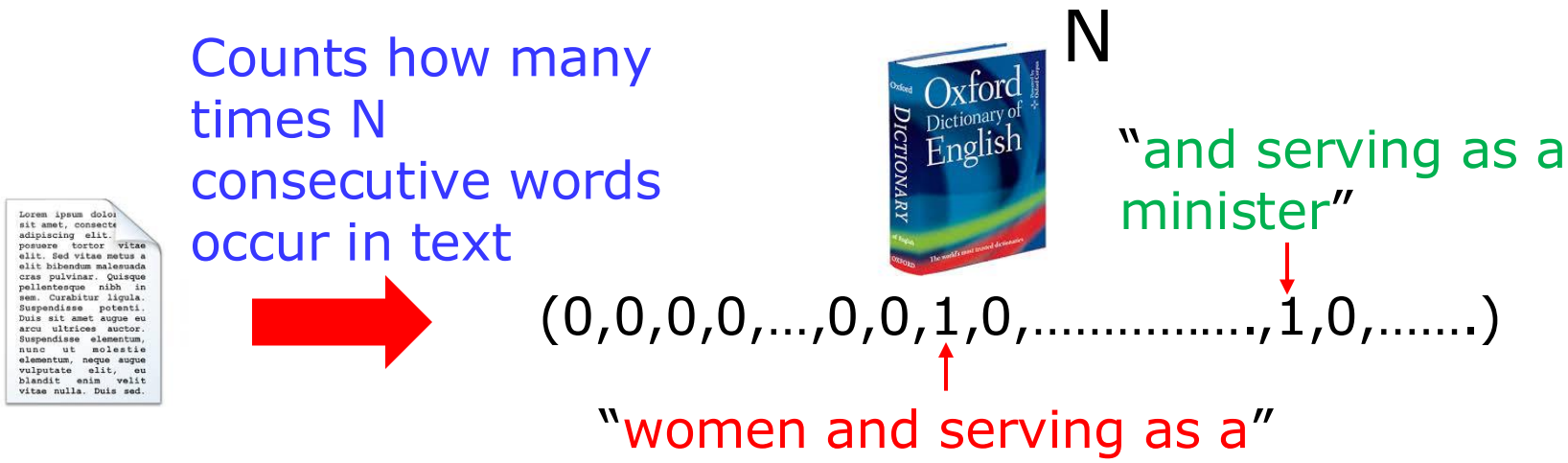
(0,1,0,0,…,0,0,1,0,……..,0,1,…….)

"valued at" "with a" "the amount"

- *"The amount is valued at Euro 47,745,533.00 with a financial institution waiting my instruction"*

Bag of Words

- N-Grams:
  - Adding order to words.
  - Group together every N consecutive words.
  - Feature for *every N-tuple* of words in dictionary.



  - *"Am a single Arab women and serving as a minister, there is a limit to my personal income and investment level"*

Bag of Words



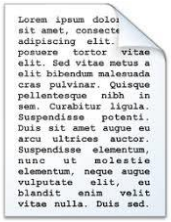
- N-Grams:
  - Adding order to words.
  - Group together every N consecutive words.
  - Feature for *every N-tuple* of words in dictionary.

- Considerations:
  - Too small N: Lose order of words and thus perhaps some of meaning of text.
  - Too large N: Texts share no N-grams. Impossible to learn anything useful if new text has no overlap in N-grams.

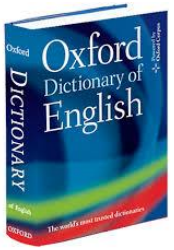
  - Remove punctuations and make everything lower-case.

Bag of Words

- Representing Bag-of-Words:
  - Quite large number of features.

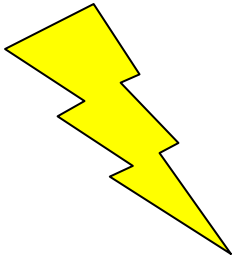






(0,1,0,0,…,2,0,1,0)

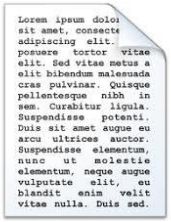
  - 171,476 words in English dictionary, so 171k features.
  - Algorithms become very slow!
  - VC-dimension issues?



Bag of Words

- Representing Bag-of-Words:
  - Quite large number of features.

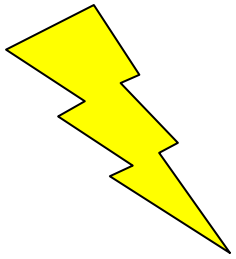






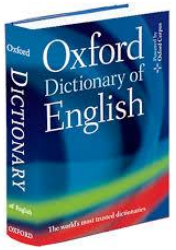
(0,1,0,0,…,0,0,1,0,……..,0,1,…….)

  - 171,476 words in English dictionary, so $171{,}476^2 \approx 29$ billion 2-grams.
  - N-grams have $171{,}476^N$ features.

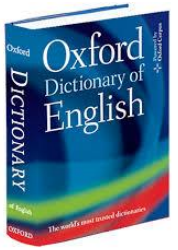


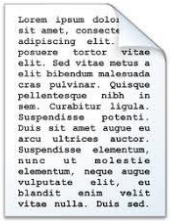
Bag of Words

- Representing Bag-of-Words:
  - Quite large number of features.



X







(0,1,0,0,…,0,0,1,0,……..,0,1,…….)

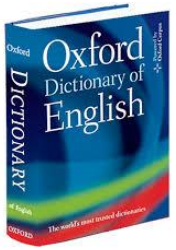
- Observation: Vectors are very sparse.

"*We are China mould let, locating in shenzhen of china.*"

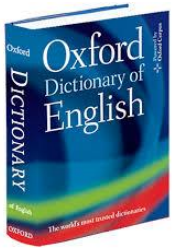
- If text has m words, then (m-N+1) N-grams.
- Linear in text length.

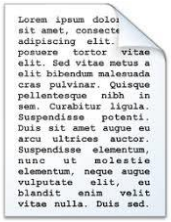
Bag of Words

- Representing Bag-of-Words:
  - Quite large number of features.



X







(0,1,0,0,…,0,0,1,0,……..,0,1,…….)

- Sparse Vector Representations:
  - 1. Ordered list of non-zero entries.
  - 2. Dictionary (hash map), mapping strings (N-grams) to counters.

Bag of Words

- Sparse Vector Representations:
  - 1. Ordered list of non-zero entries and their values.
  - 2. Dictionary (hash map), mapping strings (N-grams) to counters.

- "*We shall execute the transaction under a legitimate arrangement without breaking the law*"

(0,1,0,0,…,0,0,1,0,……,0,1,…….)

Index of coordinate

Value of coordinate

- As List: [(2,1), (34011,1), (68011,1),….].
- As Dictionary: {"we shall":1, "under a":1, "shall execute":1,…}.

N-gram

Count of N-gram

Bag of Words

- Sparse Vector Representations:

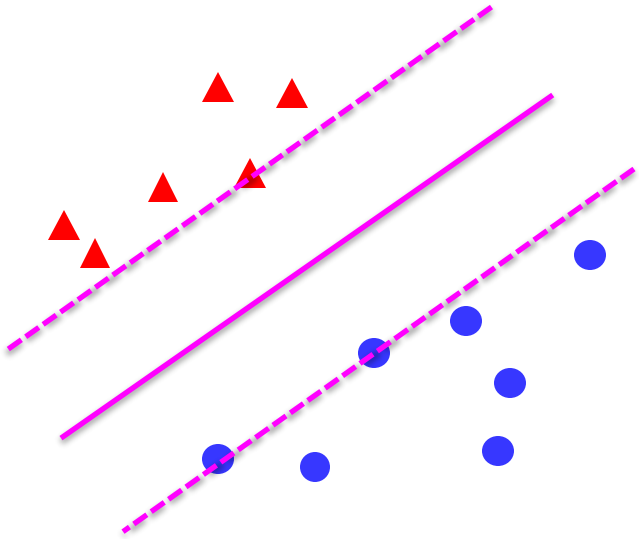
(0,1,0,0,…,0,0,1,0,……,0,1,…….)

- As List: [(2,1), (34011,1), (68011,1),….].
- As Dictionary: {”we shall”:1, ”under a”:1, “shall execute”:1,…}.

- But Wait:
  - Now we again have a representation that is not strictly a vector. Do the learning algorithms break?
  - Not all of them



Support Vector Machines



- Kernel SVM formulation:
  - $\max_{\alpha:\ \alpha_i \geq 0} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $b = -\frac{1}{2} \left(\min_{i: y_i = 1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j) + \max_{i: y_i = -1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j) \right)$
- Prediction on x:
  - $\text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i K(x_i, x) + b \right)$
- Only need to compute inner products $K(x_i, x_j)$
  - Inner products are fast to compute with sparse representations!

Bag of Words

- Sparse Vector Representations:

(0,1,0,0,…,0,0,1,0,……,0,1,…….)

- As List: [(2,1), (34011,1), (68011,1),….].
- As Dictionary: {”we shall”:1, ”under a”:1, “shall execute”:1,…}.

- Computing Inner Products:
  - Can run SVM if can compute inner products.

$(0,1,0,0,\ldots,0,0,1,0,\ldots\ldots\ldots\ldots,0,1,\ldots\ldots)^{T}(0,2,1,0,\ldots,1,0,0,0,\ldots\ldots\ldots\ldots,0,0,\ldots\ldots)$

  - [(2,1), (34011,1), (68011,1),….]
  - [(2,2), (3,1), (1581,1), (34009,1),….]
  - Scan from front to end…

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



Same coordinate!

innerProduct=0

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



Same coordinate!

innerProduct=2

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



different coordinates!

innerProduct=2

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



different coordinates!

innerProduct=2

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



Same coordinate!

innerProduct=2

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



Same coordinate!

innerProduct=3

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



different coordinates!

innerProduct=3

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Example:

$$(0,1,0,0,0,0,1,0,1)^T(0,2,1,0,0,1,1,1,0)$$

  - a=[(2,1), (7,1), (9,1)]



innerProduct=3

  - b=[(2,2), (3,1), (6,1),(7,1),(8,1)]



Return innerProduct

- Inner product:
  - Scan the two from front to end, always examining the element with smallest index.
  - Keep a running sum of the inner product.
  - Linear time in |a|+|b|.

Bag of Words

- Sparse Vector Representations:

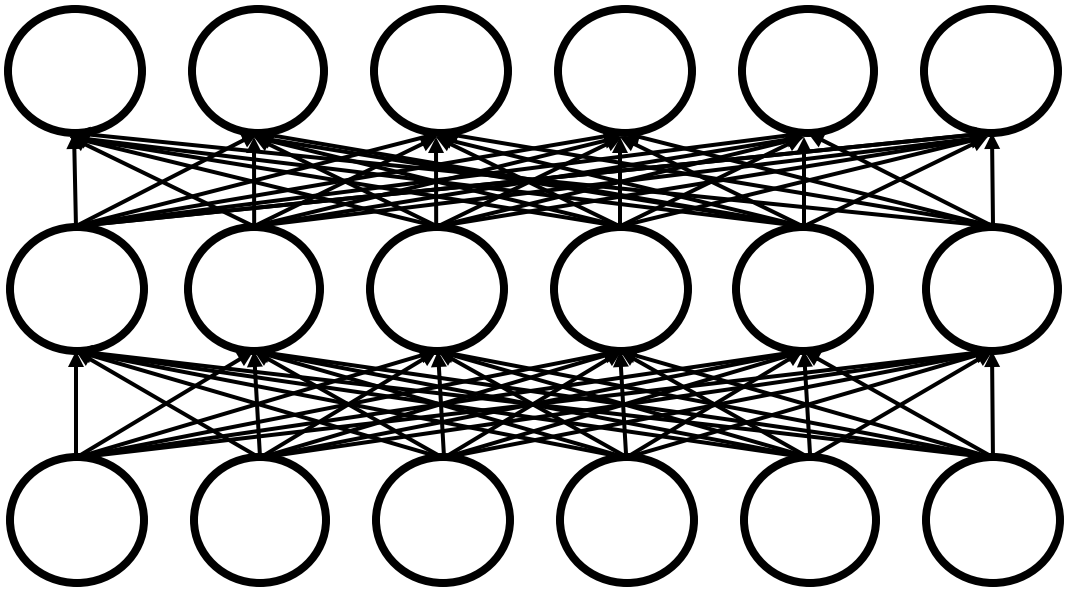
(0,1,0,0,…,0,0,1,0,……,0,1,…….)

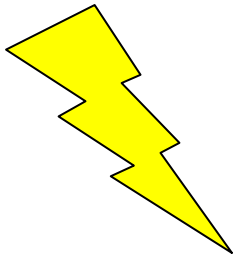
- As List: [(2,1), (34011,1), (68011,1),….].
- As Dictionary: {”we shall”:1, ”under a”:1, “shall execute”:1,…}.

- Computing Inner Products:
  - Lists: |a|+|b| time.
  - Dictionaries: Exercise to do it in time proportional to shortest list min{|a|,|b|}.

Neural Nets

- Sparse input representation does not help:





TF-IDF

Wikipedia article

AARHUS
UNIVERSITY

Machine Learning on Text

- Spam Filter:

Greetings,

I seek for a trusted, sincere and honest person to judiciously set up a charity foundation under my sponsorship.

The foundation will be strictly for helping the poorest of the poor; especially the orphanage homes and homes for destitute and the mentally retarded.

I am 76 years old and suffering from Protracted Cancer Of The Lungs and on admission at the specialist hospital intensive care unit.

My lawyer that was supposed to handle the process of executing my Will & Testament was shot during the protest against the presidential election.

If t touched your spirit to be part in putting smile and new life on the vulnerable, kindly reply through my personal below email for more information.

Mrs. Matsushita

Email: matsumorand@gmail.com

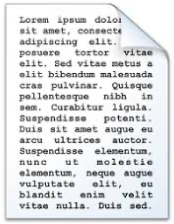
Formulaire sans titre

REMPLIR LE FORMULAIRE



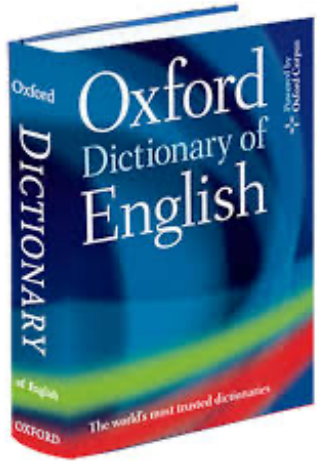
Bag of Words

- Goal: Represent text as a vector.

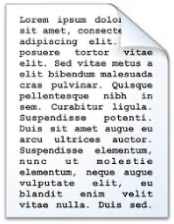


"Viagra Oral Jelly is a remedy that helps to control and restore potency during sexual arousal. The main advantage of Viagra Oral Jelly is its fast action"

- Bag of Words:
  - Assume all words in text come from the English dictionary.



Vector has 1 entry per word in dictionary





"Viagra"

(0,1,0,0,…,2,0,1,0)

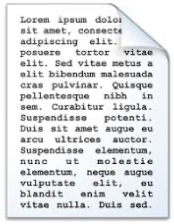
"potency" "cow"

Entry stores count of many times word occurs in text.



Bag of Words

- Goal: Represent text as a vector.



(0,1,0,0,…,2,0,1,0)

- Bag of Words:
  - Assume all words in text come from the English dictionary.
  - Index pr word, storing count of occurrences in text.

- Observations:
  - Common words like ”the, a, or, and,…” have many occurrences in most documents. Large weights.
  - They are mostly irrelevant for classification.

Inner products between documents “noised” by these words.

Large weights may give more emphasis by algorithms.

TF-IDF



- Inverse Document Frequency IDF:
  - Some words occur in almost all documents:
    - ”the, a, or, and,…”
  - Some words occur in few document:
    - ”viagra, investment, charity,…”

- Intuition: Rare words probably have more meaning as to what the document is about than common words.

- Idea: Weigh terms (=words) by how frequently they occur in documents.

TF-IDF



- Inverse Document Frequency IDF:
  - Some words occur in almost all documents:
    - ”the, a, or, and,…”
  - Some words occur in few document:
    - ”viagra, investment, charity,…”

- For collection of documents $d_1,\ldots,d_n$ (think the training documents) and a term t (a word):
  - Document frequency (fraction of documents with t)
    - $df(t) = \frac{|\{d_i : t \in d_i\}|}{n}$
  - Inverse document frequency
    - $idf(t) = \ln\left(\frac{1}{df(t)}\right) = \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$

TF-IDF



- Inverse Document Frequency IDF:
  - $idf(t) = \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$
  - $idf(\text{"minister"}) = \ln\left(\frac{3}{1}\right) \approx 1.1$
  - $idf(\text{"is"}) = \ln\left(\frac{3}{2}\right) \approx 0.4$
  - $idf(\text{"a"}) = \ln\left(\frac{3}{3}\right) = 0$
- d_1=*"Am a single Arab women and serving as a minister, there is a limit to my personal income and investment level"*
- d_2=*"The amount is valued at Euro 47,745,533.00 with a financial institution waiting my instruction"*
- d_3=*"We shall execute the transaction under a legitimate arrangement without breaking the law"*

TF-IDF



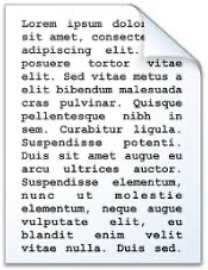
- Inverse document frequency:
  - $idf(t) = \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$
- Observations:
  - Words occurring in every document have smallest idf(t) of 0.
  - Words occurring in a single document have largest idf(t)=ln(n).
- Puts emphasis on rare words.



TF-IDF



- Term-Frequency TF:
  - For document d and term t, occurrences:
    - $f_{t,d}$ = #occurrences of t in d.
  - Bag-of-words representation of d:
    - Feature vector entry t stores $f_{t,d}$

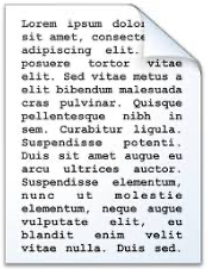


- Observation:
  - Long documents have ”heavy” vectors compared to short documents.
  - Topic of document not (not always) related to length.

TF-IDF



- Term-Frequency TF:
  - For document d and term t, occurrences:
    - $f_{t,d}$ = #occurrences of t in d.
  - Idea: ”Normalize” frequencies.
  - Replace by the term-frequency $tf(t,d) = f_{t,d}/|d|$.



“We shall execute the transaction under a legitimate arrangement without breaking the law”

- tf(”the”,d)=2/13
- tf(”under”,d)=1/13.

TF-IDF



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$
- Tf-idf:
  - Entry t in vector for document d is:
    - $tf(t,d) \cdot idf(t) = \frac{f_{t,d}}{|d|} \cdot \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$

Words occurring often in document have large weights.

Rare words are weighed higher than common words.

TF-IDF



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$
- Tf-idf:
  - Entry t in vector for document d is:
    - $tf(t,d) \cdot idf(t) = \frac{f_{t,d}}{|d|} \cdot \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$

- In Practice:
  - Tf-idf sometimes better, but no guarantees.
  - Can try both and use validation set.

TF-IDF



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$
- Tf-idf:
  - Entry t in vector for document d is:
    - $tf(t,d) \cdot idf(t) = \frac{f_{t,d}}{|d|} \cdot \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$
- Alternative:
  - Sometimes use $tf(t,d) = \ln(1 + f_{t,d})$ instead.
    - 0 when $f_{t,d}=0$.
    - Logarithmic scaling on occurrences.
  - Wiki typo: writes $1 + \ln(f_{t,d})$ instead of $\ln(1 + f_{t,d})$.



Feature Hashing

Machine Learning on Text

- Spam Filter:

Greetings,

I seek for a trusted, sincere and honest person to judiciously set up a charity foundation under my sponsorship.

The foundation will be strictly for helping the poorest of the poor; especially the orphanage homes and homes for destitute and the mentally retarded.

I am 76 years old and suffering from Protracted Cancer Of The Lungs and on admission at the specialist hospital intensive care unit.

My lawyer that was supposed to handle the process of executing my Will & Testament was shot during the protest against the presidential election.

If t touched your spirit to be part in putting smile and new life on the vulnerable, kindly reply through my personal below email for more information.

Mrs. Matsushita

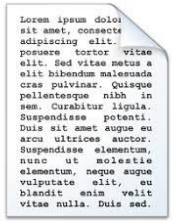
Email: matsumorand@gmail.com

Formulaire sans titre

REMPLIR LE FORMULAIRE

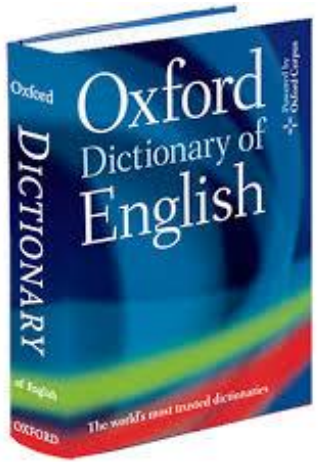
Bag of Words

- Goal: Represent text as a vector.

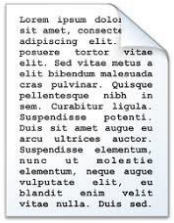


"Viagra Oral Jelly is a remedy that helps to control and restore potency during sexual arousal. The main advantage of Viagra Oral Jelly is its fast action"

- Bag of Words:
  - Assume all words in text come from the English dictionary.



Vector has 1 entry per word in dictionary





"Viagra"

(0,1,0,0,…,2,0,1,0)

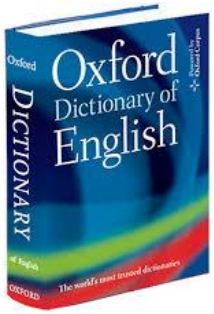
"potency" "cow"

Entry stores count of many times word occurs in text.

TF-IDF



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$ = #occurrences of t in d
- Tf-idf:
  - Entry t in vector for document d is:
    - $tf(t,d) \cdot idf(t) = \frac{f_{t,d}}{|d|} \cdot \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$

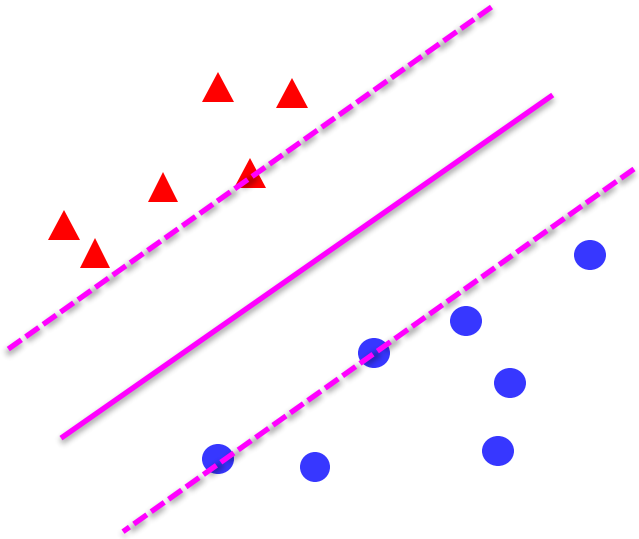


- Issues: Very long but sparse vectors.
  - Sparse Vector Representations:

(0,1,0,0,…,0,0,1,0,……..,0,1,…….)

  - As List: [(2,1), (34011,1), (68011,1),….].
  - As Dictionary: {”we shall”:1, ”under a”:1, “shall execute”:1,…}.

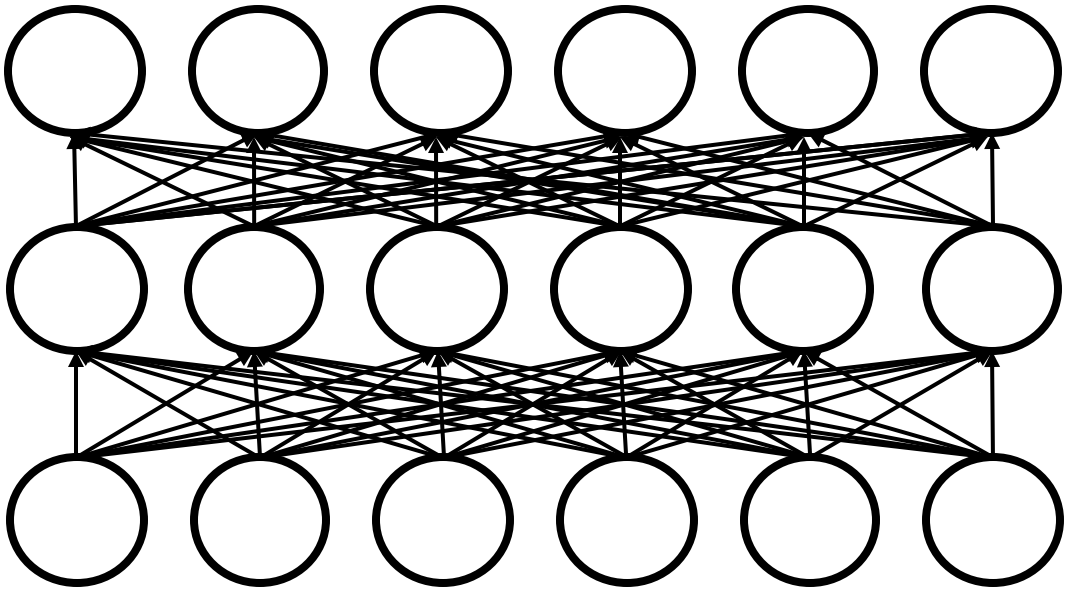
Support Vector Machines

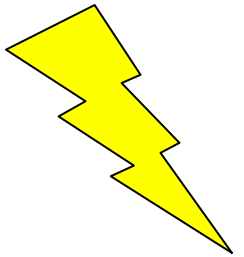


- Kernel SVM formulation:
  - $\max_{\alpha:\ \alpha_i \geq 0} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $b = -\frac{1}{2}\left(\min_{i:y_i=1} \sum_{\alpha_j \neq 0} \alpha_j y_j\, K(x_i, x_j) + \max_{i:y_i=-1} \sum_{\alpha_j \neq 0} \alpha_j y_j\, K(x_i, x_j)\right)$
- Prediction on x:
  - $\text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i\, K(x_i, x) + b\right)$
- Only need to compute inner products $K(x_i, x_j)$
  - Inner products are fast to compute with sparse representations!

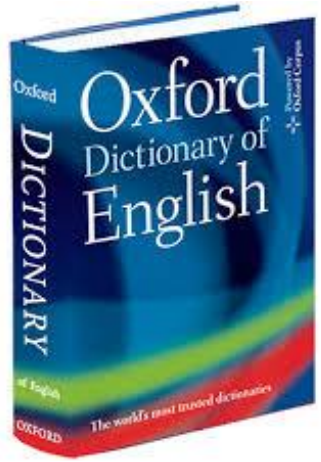
Neural Nets

- Sparse input representation does not help:



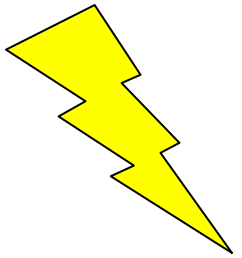


Bag of Words



- Reducing #Features:
  - Start by scanning all training documents.
  - Collect all words/N-grams and give them consecutive numbers 0,1,2,… (e.g. in hash map/dictionary)
  - Only have an entry in feature vector per word seen in training data.

- Observations:
  - Reduces #features somewhat.
  - Still proportional to total length of all training documents.



Feature Hashing

- Very Sparse Feature Vectors:

  (0,1,0,0,…,0,0,1,0,……..,0,1,…….)

- Each entry corresponds to a word or N-gram in text.

- Feature Hashing:
  - Map feature vector to a much shorter vector.
  - k features rather than d, with k<<d.
  - Dimensionality reduction that is fast to compute for sparse vectors.

Feature Hashing

Notation
$[d]=\{0,\ldots,d-1\}$

- Feature Hashing:

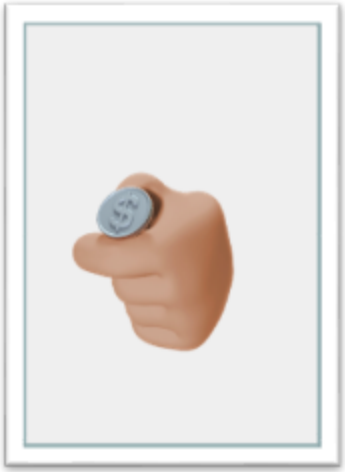
$$u = (0,1,0,0,0,0,0,1,0,1,2,0,0,1,0,0,0,1,0) \quad (d)$$

$$f(u) = (0,0,0,0,0,0) \quad (k)$$

- Approach:
  - Hash function $h : [d] \to [k]$. Maps features in u to features in new vector f(u).
  - Hash function $g : [d] \to \{-1,1\}$. Maps features in u to a sign.
  - If $u_t=c$, then $f(u)_{h(t)} \mathrel{+}= c \cdot g(t)$.

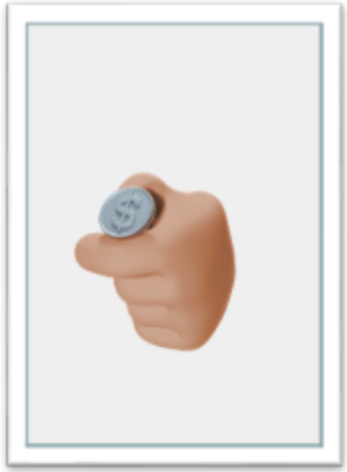
Hash Functions

- Hash Functions
  - Basic idea:
    - Functions that map from large domain [d] to small domain [k].
    - Should distribute items in [d] ”evenly” in [k].
  - A bit more formally:
    - Hash functions are *chosen* using randomness.
    - Example: Fix a prime p>d.
      - Choose *a* uniformly in {1,…,p-1}.
      - Choose *b* uniformly in {0,…,p-1}.
      - Use hash function $h_{a,b}(x)=((ax+b) \bmod p) \bmod k$.
    - $Pr_{a,b}[h_{a,b}(x)=h_{a,b}(y)] \leq 1/k$ for any $x \neq y$.



Hash Functions

- Hash Functions
  - Basic idea:
    - Functions that map from large domain [d] to small domain [k].
    - Should distribute items in [d] ”evenly” in [k].



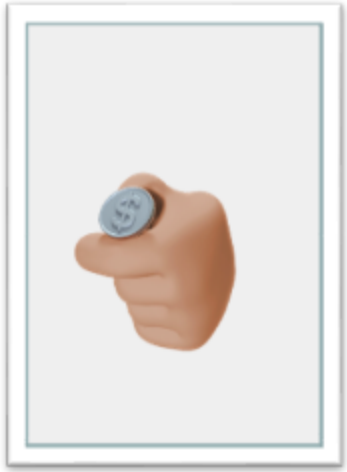
  - A bit more formally:
    - Hash functions are *chosen* using randomness.
    - Often use very little randomness, such as just choosing a and b in

      $$h_{a,b}(x)=((ax+b) \bmod p) \bmod k$$

    - After choosing the function, it is deterministic, i.e.
      - $h_{a,b}(x)$ remains the same on later evaluations.

Hash Functions

- Hash Functions
  - Basic idea:
    - Functions that map from large domain [d] to small domain [k].
    - Should distribute items in [d] ”evenly” in [k].



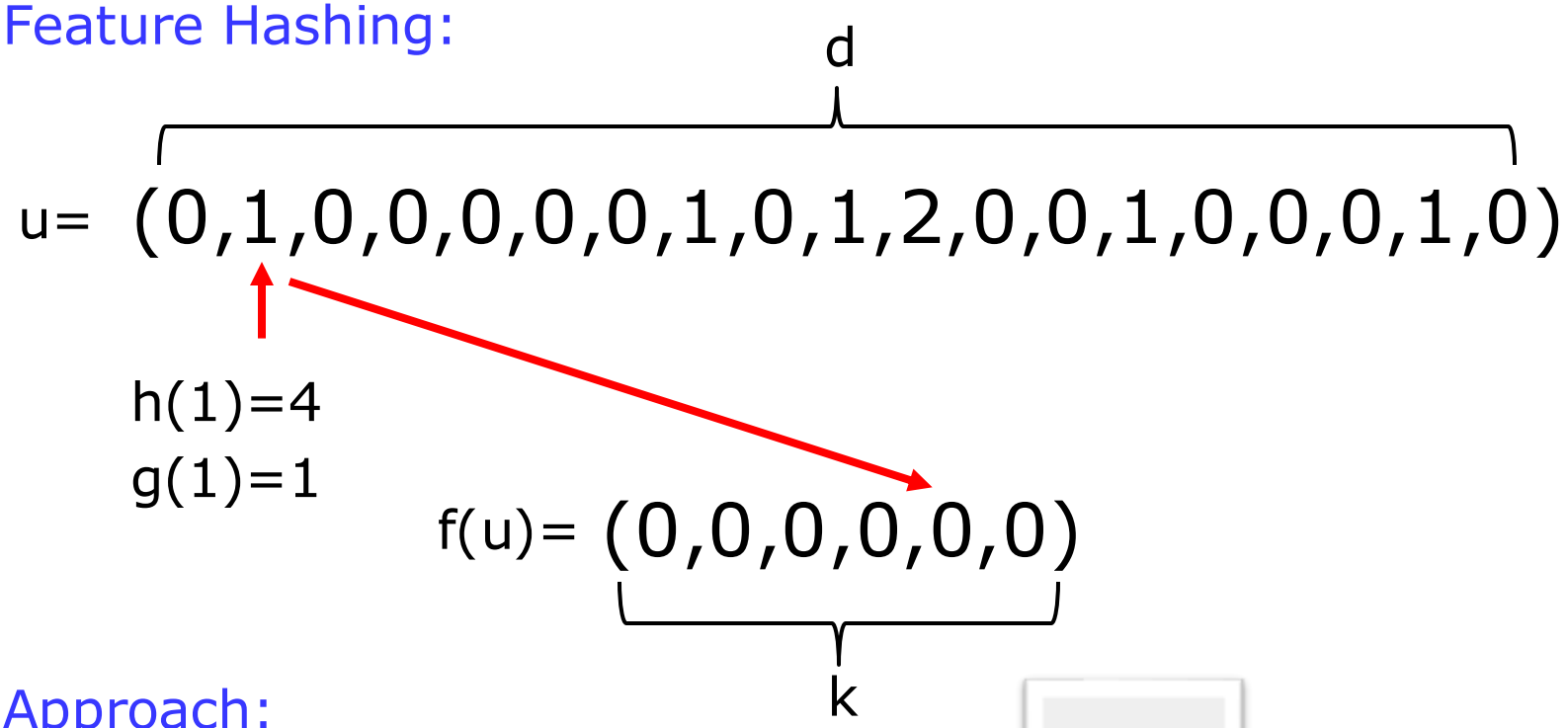
  - We need:
    - There are simple hash functions with:
      - Small chance of collision: $Pr_h[h(x)=h(y)]=1/k$.
      - The hash function $g : [d] \rightarrow \{-1,1\}$ has $g(x)$ and $g(y)$ independent and uniform random for any $x \neq y$.

Hash Functions

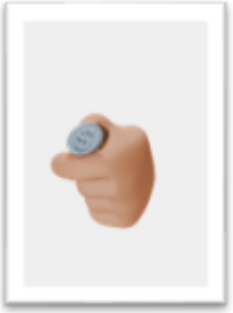
- Hash Functions
  - Huge literature on hash functions.
  - Follow a ”Randomized Algorithms” course to learn more.

Feature Hashing

- Feature Hashing:

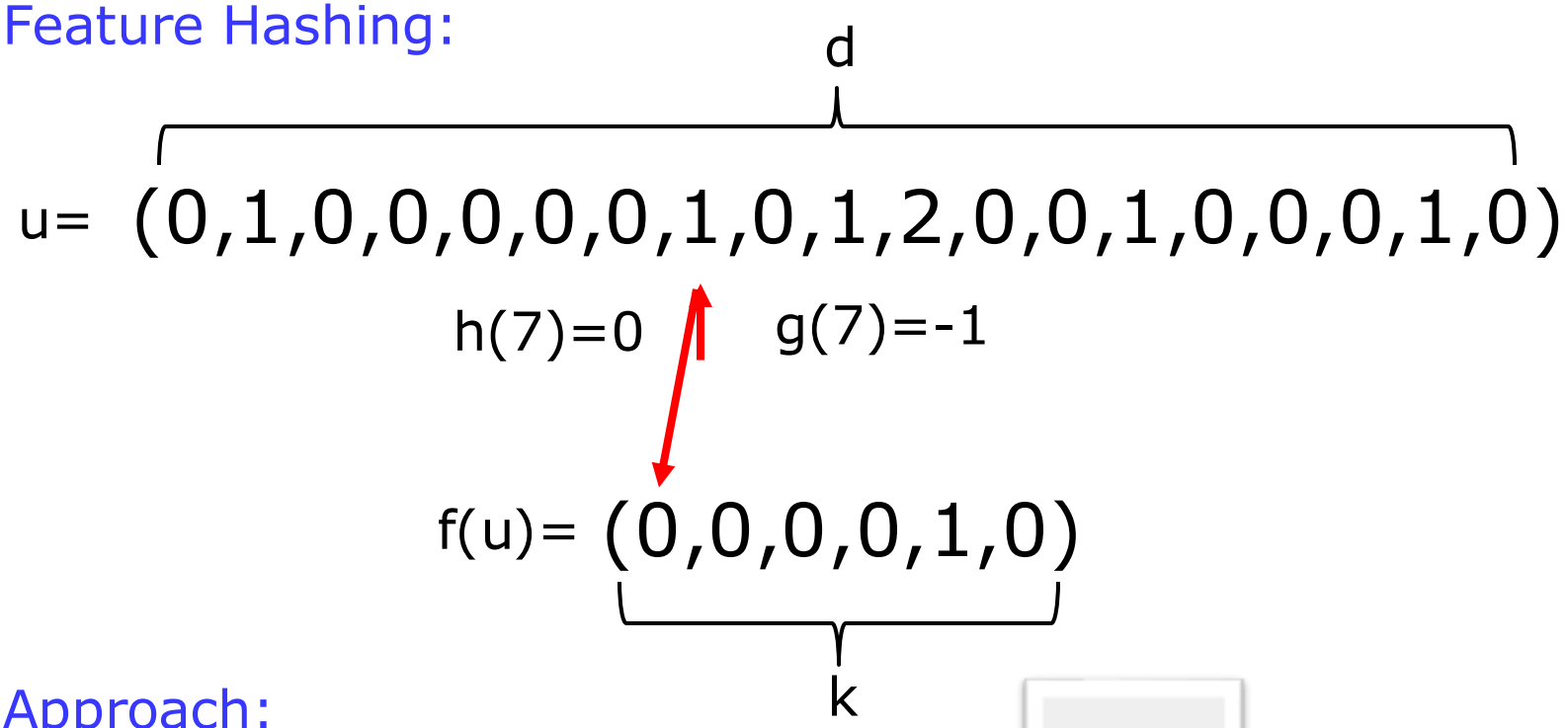


- Approach:
  - Hash function h : [d] → [k].
  - Hash function g : [d] → {-1,1}.
  - If u_t=c, then $f(u)_{h(t)}$+= c·g(t).

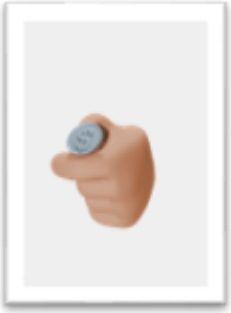


Feature Hashing

- Feature Hashing:

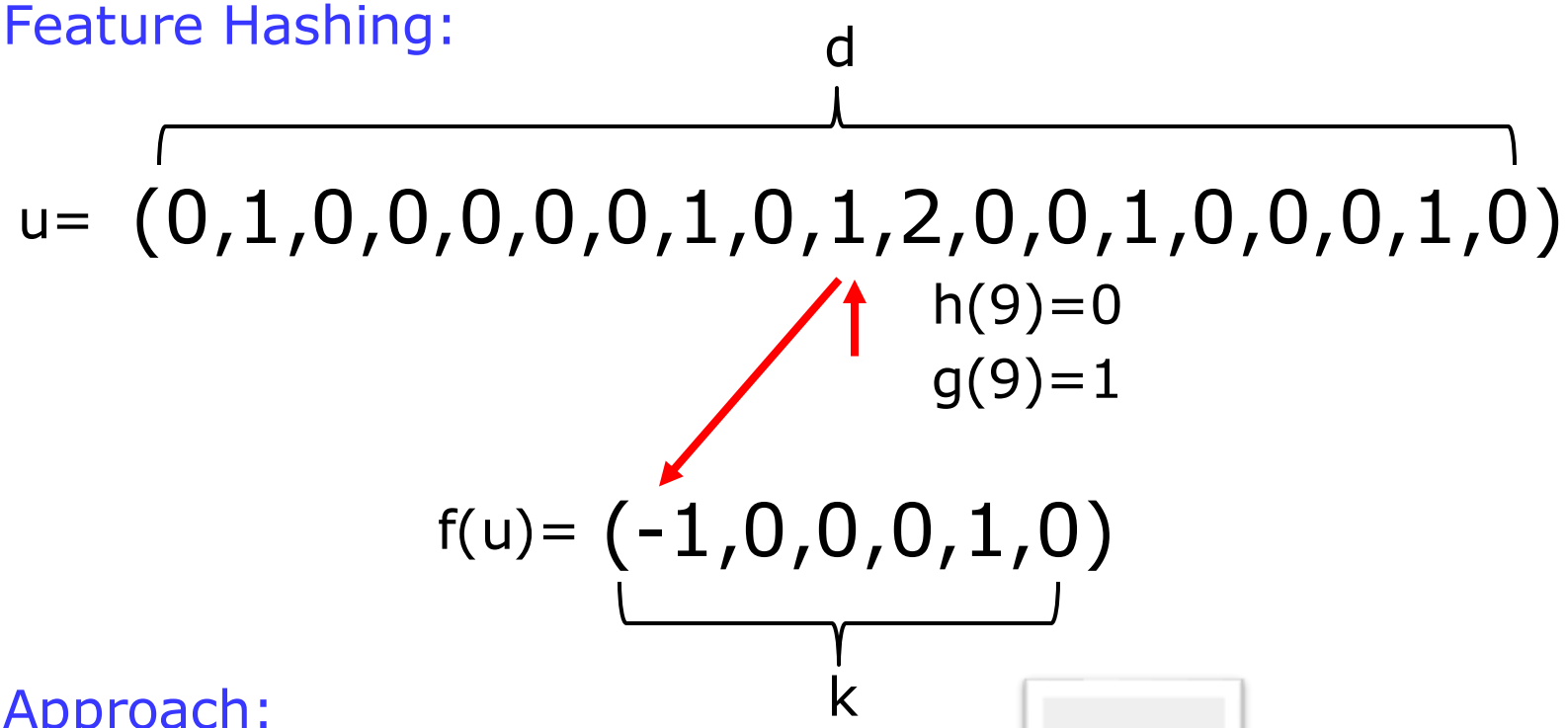


- Approach:
  - Hash function h : [d] → [k].
  - Hash function g : [d] → {-1,1}.
  - If u_t=c, then $f(u)_{h(t)}$+= c·g(t).

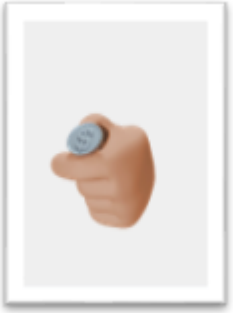


Feature Hashing

- Feature Hashing:

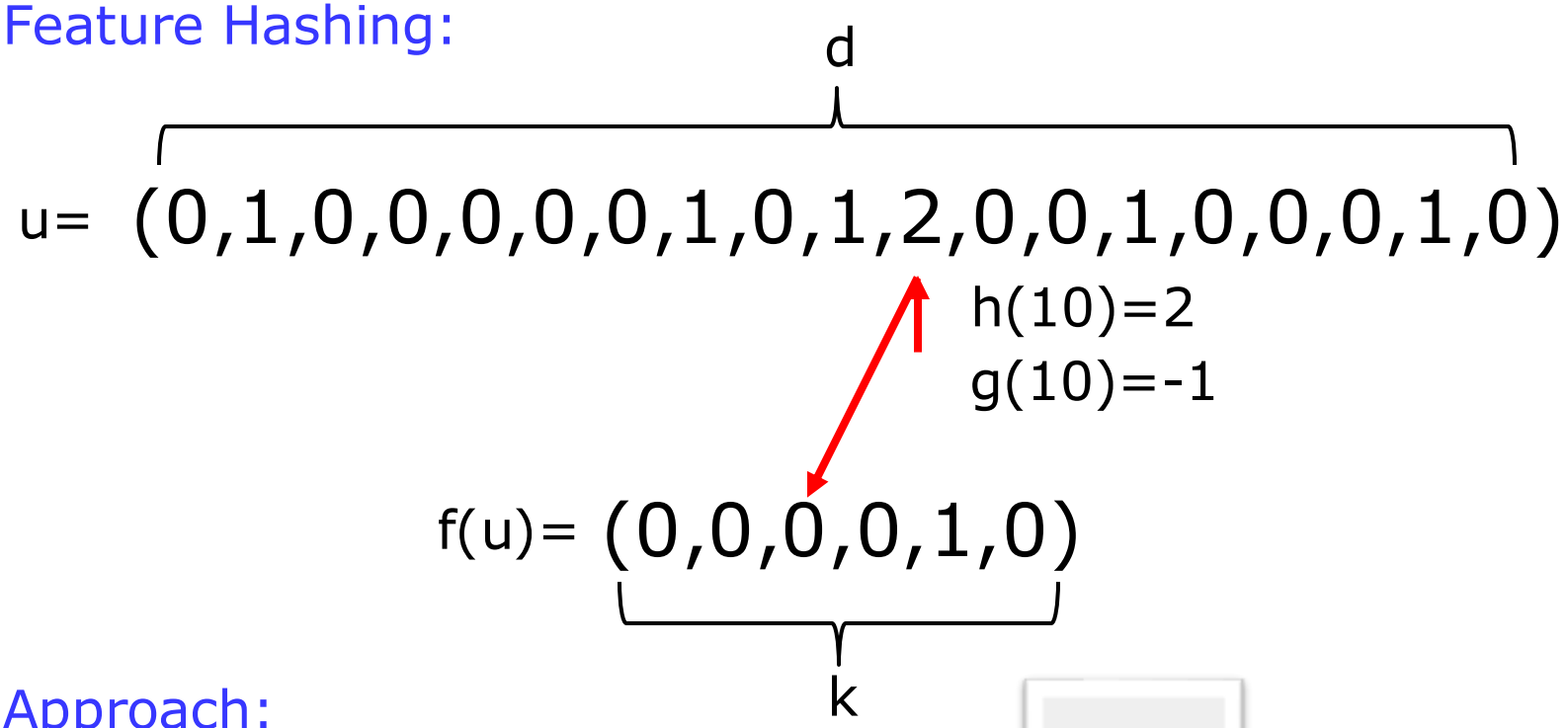


- Approach:
  - Hash function h : [d] → [k].
  - Hash function g : [d] → {-1,1}.
  - If u_t=c, then $f(u)_{h(t)}$+= c·g(t).

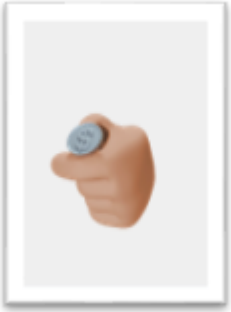


Feature Hashing

- Feature Hashing:

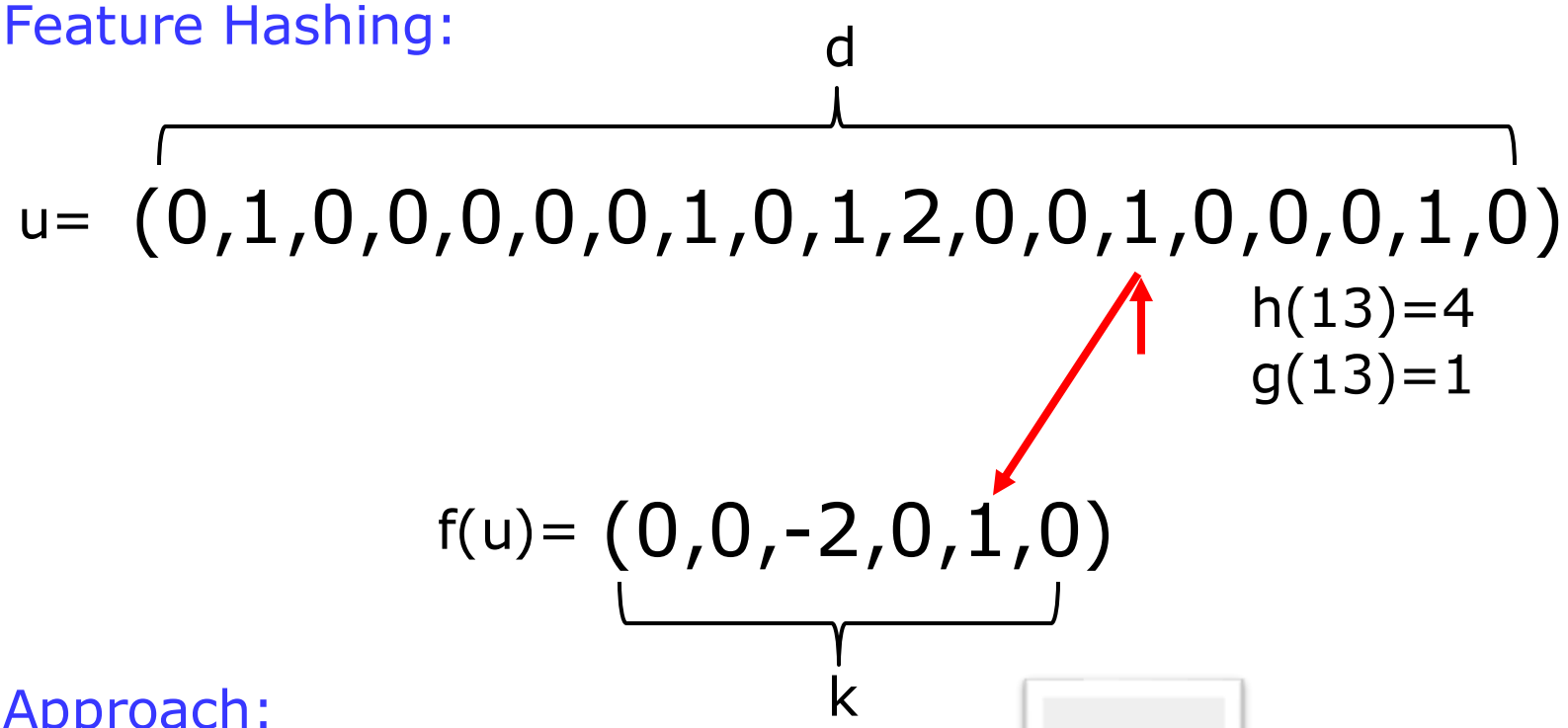


- Approach:
  - Hash function h : [d] → [k].
  - Hash function g : [d] → {-1,1}.
  - If u_t=c, then $f(u)_{h(t)}$+= c·g(t).

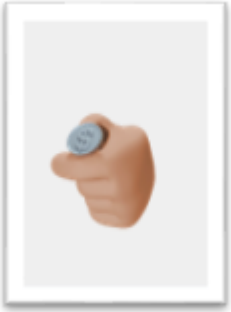


Feature Hashing

- Feature Hashing:

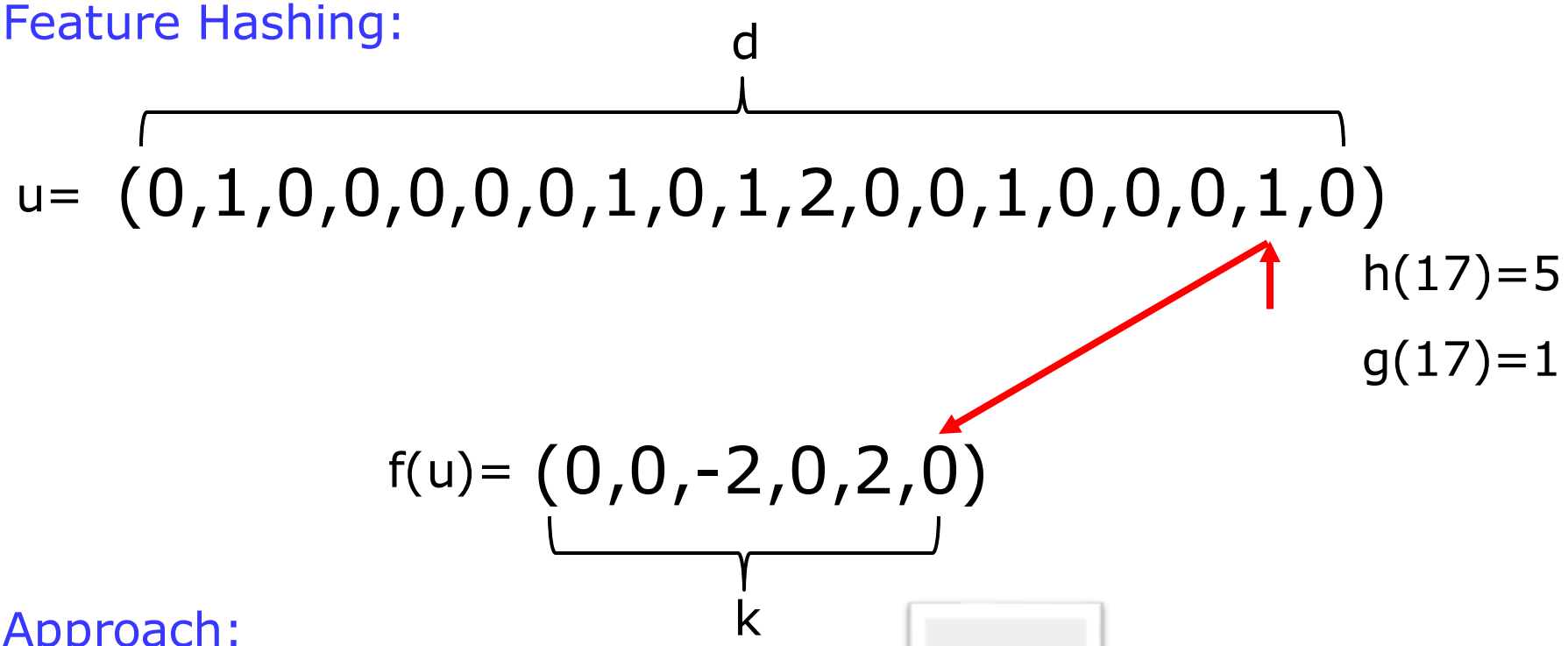


- Approach:
  - Hash function h : [d] → [k].
  - Hash function g : [d] → {-1,1}.
  - If u_t=c, then $f(u)_{h(t)}$+= c·g(t).

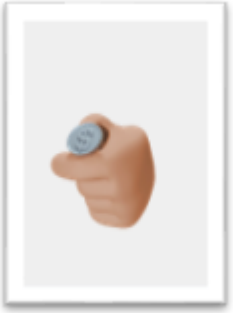


Feature Hashing

- Feature Hashing:



- Approach:
  - Hash function h : [d] → [k].
  - Hash function g : [d] → {-1,1}.
  - If u_t=c, then $f(u)_{h(t)}$+= c·g(t).



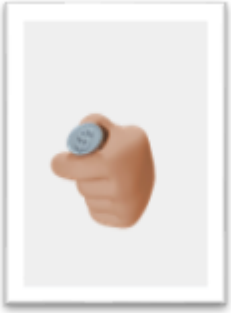
Feature Hashing

- Feature Hashing:

$$u = (0,1,0,0,0,0,0,1,0,1,2,0,0,1,0,0,0,1,0) \quad (d)$$

$$f(u) = (0,0,-2,0,2,1) \quad (k)$$

- Approach:
  - Hash function $h : [d] \to [k]$.
  - Hash function $g : [d] \to \{-1,1\}$.
  - If $u_t = c$, then $f(u)_{h(t)} \mathrel{+}= c \cdot g(t)$.



Feature Hashing

- Feature Hashing:

u= (0,1,0,0,0,0,0,1,0,1,2,0,0,1,0,0,0,1,0)



f(u)= (0,0,-2,0,2,1)

f(v)= (0,0,-1,0,0,0)

h(10)=2

g(10)=-1



v= (0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0)

- Crucial: Same hash function used on all documents/vectors.

Feature Hashing

Coordinate j contributes $u_j v_j$ to inner product.

End in same bucket with h.

Contribute $g(j)u_j g(j)v_j = u_j v_j$ after hashing.



- Feature Hashing:

u= (0,1,0,0,0,0,0,1,0,1,2,0,0,1,0,0,0,1,0)

f(u)= (0,0,-2,0,2,1)

f(v)= (0,0,-1,0,0,0)

h(10)=2

g(10)=-1

v= (0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0)

- Crucial: Same hash function used on all documents/vectors.

Feature Hashing

- Feature Hashing:

Coordinate i and j contribute 0 to inner product when $i \neq j$.

May end in same bucket with h.

Contribute $g(i)u_i g(j)v_j$. If signs independent, then *expect* 0.



$u = (0,1,0,0,0,0,0,1,0,1,2,0,0,1,0,0,0,1,0)$

$f(u) = (0,0,-2,0,2,1)$

$f(v) = (0,0,-1,0,0,0)$

$v = (0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0)$

- Crucial: Same hash function used on all documents/vectors.

Feature Hashing

- Feature Hashing:

Exercise: Prove that $E[f(u)^T f(v)] = u^T v$.



$$u = (0,1,0,0,0,0,0,1,0,1,2,0,0,1,0,0,0,1,0)$$

$$f(u) = (0,0,-2,0,2,1)$$
$$f(v) = (0,0,-1,0,0,0)$$

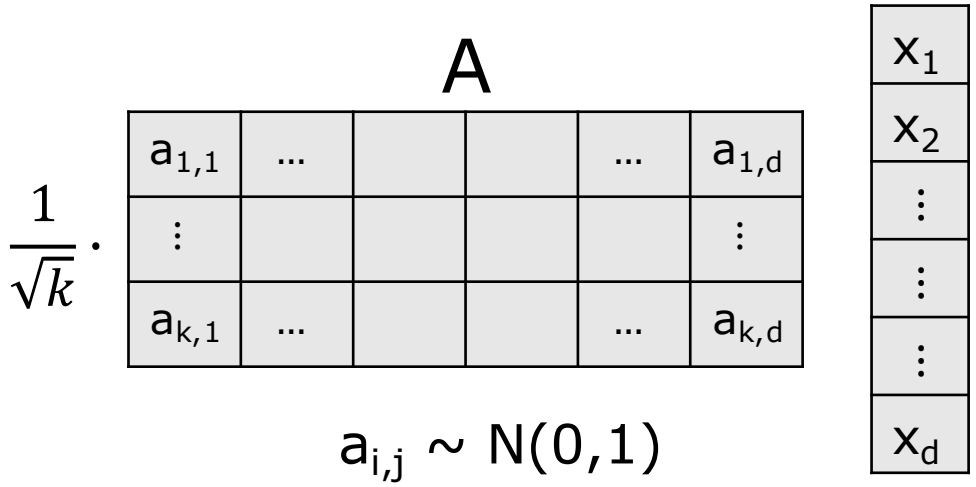
$$v = (0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0)$$

- Crucial: Same hash function used on all documents/vectors.

Random Projections

$$L = \frac{1}{\sqrt{k}} A$$

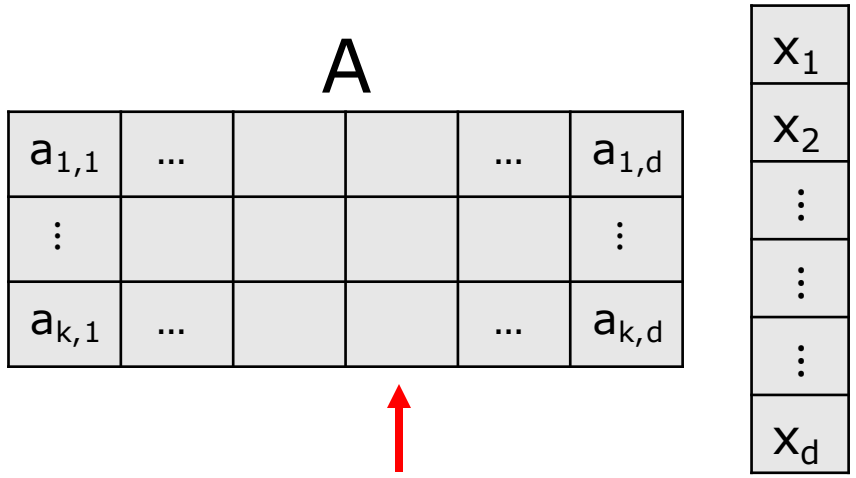
- Johnson-Lindenstrauss Transform



- Remark:
  - Could also work if choose $a_{i,j}$ uniform in {-1,1}.

Random Projections

- Feature Hashing

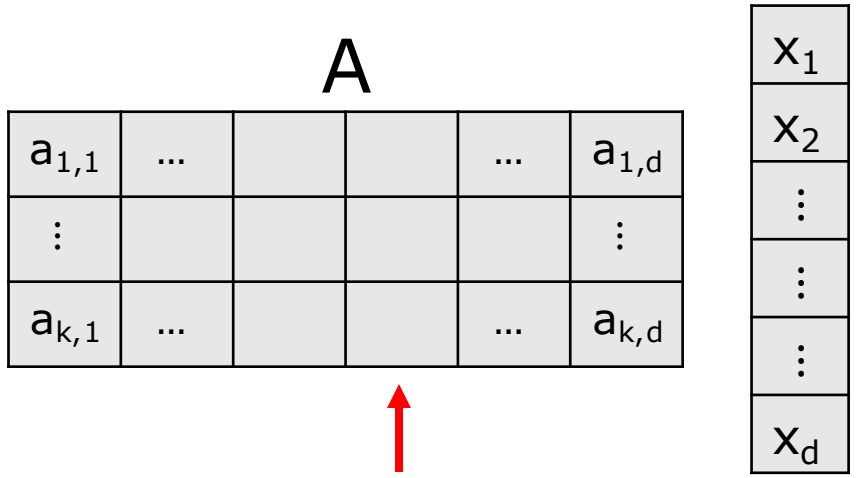


All 0, except uniform random entry, which is uniform in {-1,1}.

- Remark:
  - Position of non-zero in column j is h(j).
  - Sign of the non-zero in column j is g(j).

Random Projections

- Feature Hashing

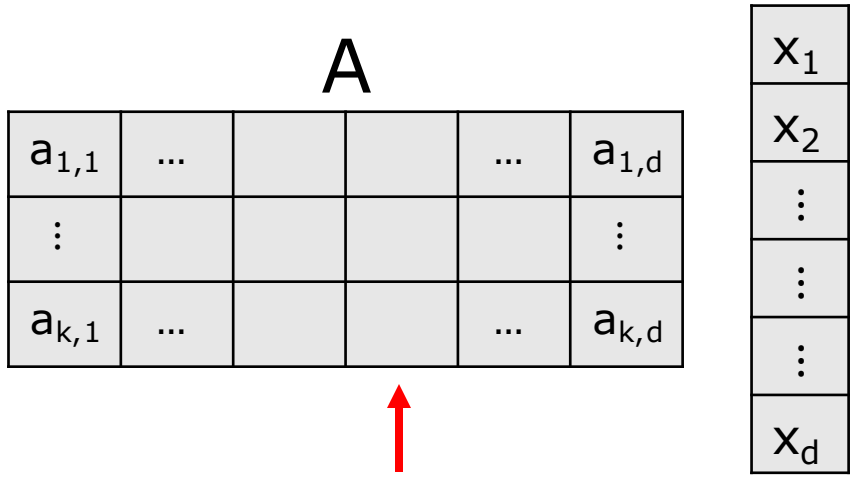


All 0, except uniform random entry, which is uniform in {-1,1}.

- Remark:
  - Feature hashing *does not* guarantee preserving all distances to within $(1 \pm \varepsilon)$ when k=$\theta(\varepsilon^{-2} \ln n)$.

Random Projections

- Feature Hashing

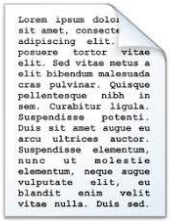


All 0, except uniform random entry, which is uniform in {-1,1}.

- Remark:
  - Feature hashing *is fast* (constant time pr. non-zero in x).
  - Johnson-Lindenstrauss *is slower* (k time pr. non-zero in x).

Feature Hashing

- Goal: Represent text as a vector.



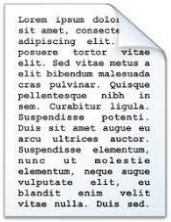
"We shall execute the transaction under a legitimate arrangement without breaking the law"

- Bag of Words with Feature Hashing:
  - Just need hash functions h and g mapping words/N-grams directly to indices in [k] and a sign {-1,1}.

- Scan text:
  - For each word t, $f(u)_{h(t)} += g(t)$.

Feature Hashing

- Goal: Represent text as a vector.



"We shall execute the transaction under a legitimate arrangement without breaking the law"

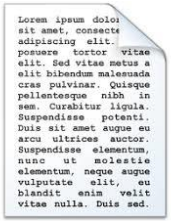
- Bag of Words with Feature Hashing:
  - For each word t, $f(u)_{h(t)} += g(t)$.

h("we")=1
g("we")=-1

$$f(u) = (0,0,0,0,0,0)$$

Feature Hashing

- Goal: Represent text as a vector.



"We shall execute the transaction under a legitimate arrangement without breaking the law"

- Bag of Words with Feature Hashing:
  - For each word t, $f(u)_{h(t)} += g(t)$.

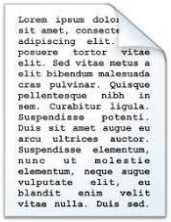
h("shall")=0

g("shall")=1

$$f(u) = (0,-1,0,0,0,0)$$

Feature Hashing

- Goal: Represent text as a vector.



"We shall execute the transaction under a legitimate arrangement without breaking the law"

- Bag of Words with Feature Hashing:
  - For each word t, $f(u)_{h(t)} += g(t)$.

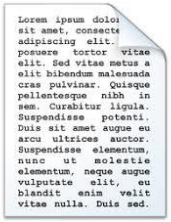
h("execute")=4

g("execute")=1

$$f(u) = (1,-1,0,0,0,0)$$

Feature Hashing

- Goal: Represent text as a vector.



"We shall execute the transaction under a legitimate arrangement without breaking the law"

- Bag of Words with Feature Hashing:
  - For each word t, $f(u)_{h(t)} += g(t)$.

$$f(u) = (1,-1,0,0,1,0)$$

Feature Hashing

- Very Sparse Feature Vectors:

  (0,1,0,0,…,0,0,1,0,……..,0,1,…….)

- Each entry corresponds to a word or N-gram in text.

- Feature Hashing:
  - Map feature vector to a much shorter vector.
  - k features rather than d, with k<<d.
  - Dimensionality reduction that is fast to compute for sparse vectors.